

Monthly Feedback Loop Model

This module implements a monthly time-series forecasting workflow with online learning and synthetic data generation.

What it does

`monthly_feedback_loop.py` performs a complete monthly feedback loop for energy production forecasting:

1. Connects to the Neon PostgreSQL database using the repo's existing `utils.db` helpers.
2. Ensures the required tables exist:
 - `gold.monthly_production`
 - `model_metrics`
3. Loads the latest row from `gold.monthly_production`.
4. Generates a new row for the next month using the previous monthly trend, seasonality, and Gaussian noise.
5. Inserts the new row into `gold.monthly_production` as real production data.
6. Loads historical monthly data and encodes month cyclically using sine/cosine features.
7. Trains an online forecasting model using `sklearn.linear_model.SGDRegressor`.
8. Trains a second explanation model using `sklearn.linear_model.SGDClassifier`.
9. Predicts the latest month production and generates a learned prediction reason.
10. Writes the prediction metrics and explanation into `model_metrics`.

Key features

- Synthetic monthly data generation for a realistic next-month production row.
- Incremental/online forecasting with `SGDRegressor`.
- Learned explanation category with `SGDClassifier`.
- Stores model metrics and prediction explanations in the database.
- Uses cyclical month encoding for seasonality.

Required environment

- Python with dependencies from the repo `requirements.txt`
- `NEON_DATABASE_URL` environment variable pointing to the Neon PostgreSQL database

How to run

From the repository root:

```
export NEON_DATABASE_URL="postgresql://<user>:<password>@<host>:  
<port>/<database>"  
python energy_pipeline/models/monthly_feedback_loop.py
```

What is saved

- `gold.monthly_production`: inserts a synthetic next month row
- `model_metrics`: stores the forecasted value, actual value, absolute error, and explanation reason

Notes

- The script assumes `gold.monthly_production` is already populated with historical rows.
- If the database is empty, the script will raise an error.
- The explanation model is lightweight and uses feature-driven reason categories (season, growth, renewable share).